

1. ROTARY ENCODER

A **rotary encoder** is a type of position sensor which is used for determining the angular position of a rotating shaft. It generates an electrical signal, either analog or digital, according to the rotational movement. Therefore, is an electromechanical device that converts rotational movement into an electrical signal.

Types of rotary encoders:

Output signal classification		Sensing technology classification		
Incremental: generates pulses to measure relative changes	Absolute: provides a unique digital code for each position, making it ideal for precise positioning even after power loss.	magnetic	Optical	laser

Operation:

A rotary encoder operates by converting the angular motion or position of a shaft into an analog or digital signal, typically using a rotating disk with a specific pattern, such as slots or magnetic poles. As the shaft rotates, this pattern is read by a stationary sensor (optical or magnetic) that generates pulses or unique binary codes. In **incremental encoders**, this produces two square wave signals (Phase A and Phase B) that are 90 degrees out of phase, this technique is known as quadrature allowing a controller to determine direction by checking which signal leads, and speed by counting the pulse frequency. **Absolute encoders** use a more complex disk pattern to generate a unique digital code for every specific position, allowing the device to report the exact angle immediately upon powering up without needing to return to a home position.

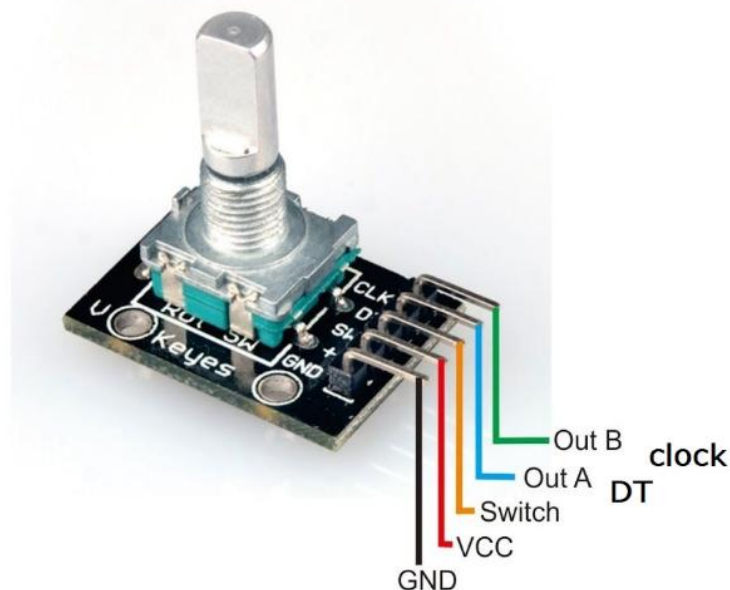
Difference between rotary encoder and potentiometer:

Feature	Rotary encoder	Potentiometer
Output signal	Digital pulses	Analog (voltage)
Rotation range	Continuous rotation (infinite)	Absolute (Knows exact position)
Position sensing	Relative (Tracks movement)	Absolute (Knows exact position)
Data representation	Speed and direction	Position/angle

Rotary encoder has 4 pins:

VCC (+): Supplies power to the encoder module (3.3V); GND: Connects to the common ground of the microcontroller; CLK (A): Outputs square wave pulses to count rotational steps; DT (B): Outputs out-of-phase pulses to determine rotation direction; SW: Outputs a signal when the knob is pressed (push-button).

With Breakout Board



Without Breakout

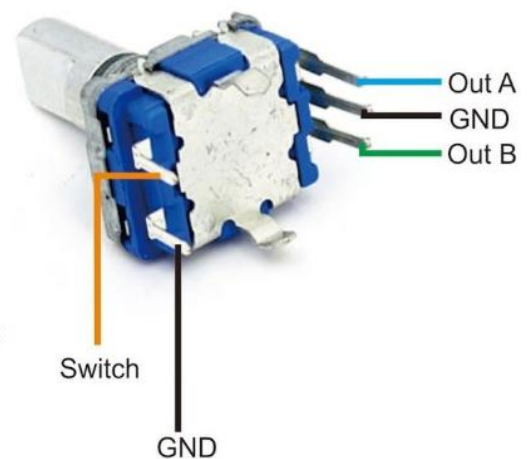


Figure 1: Rotary encoder pinout

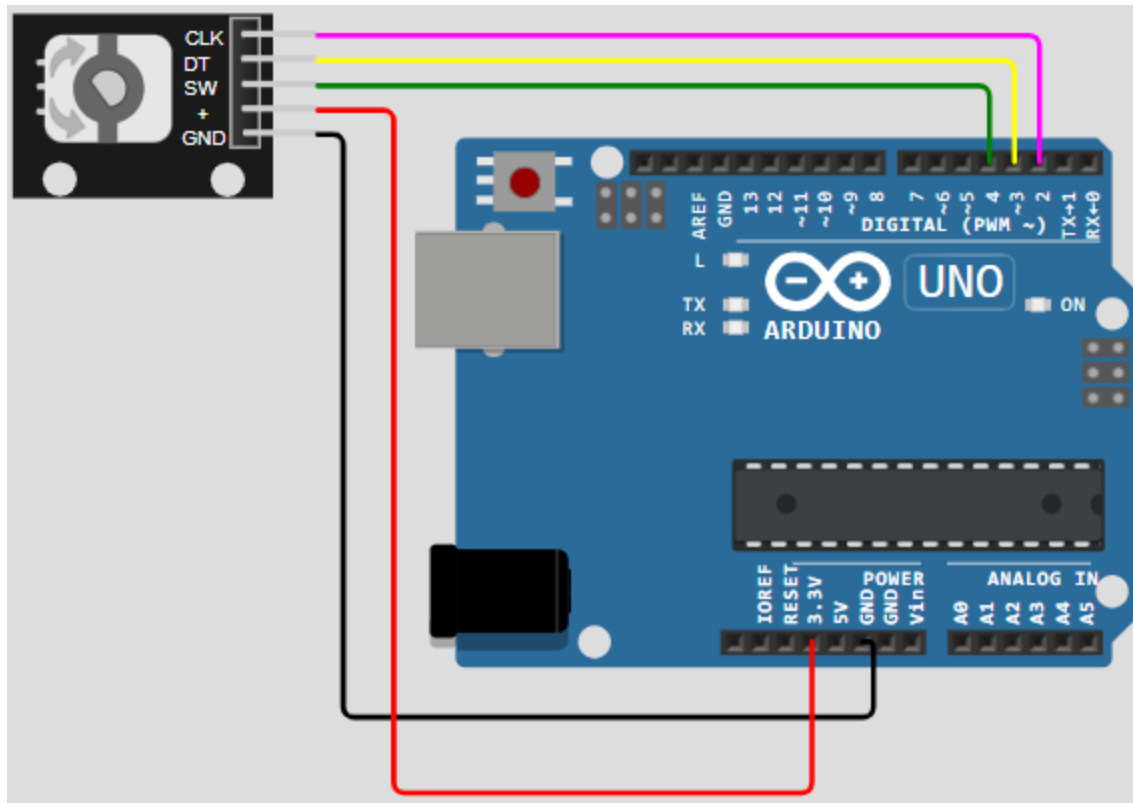


Figure 2: Circuit Diagram for a rotary encoder

I. Rotary encoder without interrupt

- Detects the amount of rotation and direction of an encoder
- 1 detent (click) in clockwise direction, adds the counter by 1.
- 1 detent (click) in ant-clockwise direction, decreases the counter by 1.
- Detects if the button is pressed

This is the polling method, which continuously checks on pin's state. This has 2 drawbacks: waste Arduino resource and some counter may be missed if another code takes long time to execute

```
#include <ezButton.h> // the library to use for SW pin
#define CLK_PIN 2
#define DT_PIN 3
#define SW_PIN 4

#define DIRECTION_CW 0 // clockwise direction
```

```
#define DIRECTION_CCW 1 // counter-clockwise direction
int counter = 0;
int direction = DIRECTION_CW;
int CLK_state;
int prev_CLK_state;
ezButton button(SW_PIN); // create ezButton object that attach to pin 4
void setup() {
  Serial.begin(9600);
  // configure encoder pins as inputs
  pinMode(CLK_PIN, INPUT);
  pinMode(DT_PIN, INPUT);
  button.setDebounceTime(50); // set debounce time to 50 milliseconds
  // read the initial state of the rotary encoder's CLK pin
  prev_CLK_state = digitalRead(CLK_PIN);
}
void loop() {
  button.loop(); // MUST call the loop() function first
  // read the current state of the rotary encoder's CLK pin
  CLK_state = digitalRead(CLK_PIN);

  // If the state of CLK is changed, then pulse occurred
  // React to only the rising edge (from LOW to HIGH) to avoid double count
  if (CLK_state != prev_CLK_state && CLK_state == HIGH) {
    // if the DT state is HIGH
    // the encoder is rotating in counter-clockwise direction => decrease the counter
    if (digitalRead(DT_PIN) == HIGH) {
      counter--;
      direction = DIRECTION_CCW;
    } else {
      // the encoder is rotating in clockwise direction => increase the counter
      counter++;
    }
  }
}
```

```
    direction = DIRECTION_CW;
}
Serial.print("DIRECTION: ");
if (direction == DIRECTION_CW)
    Serial.print("Clockwise");
else
    Serial.print("Counter-clockwise");

Serial.print(" | COUNTER: ");
Serial.println(counter);
}
// save last CLK state
prev_CLK_state = CLK_state;

if (button.isPressed()) {
    Serial.println("The button is pressed");
}
}
```

II. Rotary encoder with interrupt

- This uses interrupts which solves polling method disadvantage. Interrupts eliminate the need for constant checking of a particular event. This allows a microcontroller to carry out other tasks without overlooking an event

```
#include <ezButton.h> // the library to use for SW pin

#define CLK_PIN 2
#define DT_PIN 3
#define SW_PIN 4

#define DIRECTION_CW 0 // clockwise direction
#define DIRECTION_CCW 1 // counter-clockwise direction

volatile int counter = 0;
volatile int direction = DIRECTION_CW;
volatile unsigned long last_time; // for debouncing
int prev_counter;

ezButton button(SW_PIN); // create ezButton object that attach to pin 4

void setup() {
  Serial.begin(9600);

  // configure encoder pins as inputs
  pinMode(CLK_PIN, INPUT);
  pinMode(DT_PIN, INPUT);
  button.setDebounceTime(50); // set debounce time to 50 milliseconds

  // use interrupt for CLK pin is enough
  // call ISR_encoderChange() when CLK pin changes from LOW to HIGH
  attachInterrupt(digitalPinToInterrupt(CLK_PIN), ISR_encoderChange, RISING);
```

```
}

void loop() {
  button.loop(); // MUST call the loop() function first

  if (prev_counter != counter) {
    Serial.print("DIRECTION: ");
    if (direction == DIRECTION_CW)
      Serial.print("Clockwise");
    else
      Serial.print("Counter-clockwise");

    Serial.print(" | COUNTER: ");
    Serial.println(counter);

    prev_counter = counter;
  }

  if (button.isPressed()) {
    Serial.println("The button is pressed");
  }

  // TO DO: your other work here
}

void ISR_encoderChange() {
  if ((millis() - last_time) < 50) // debounce time is 50ms
    return;

  if (digitalRead(DT_PIN) == HIGH) {
    // the encoder is rotating in counter-clockwise direction => decrease the counter
```

```
    counter--;  
    direction = DIRECTION_CCW;  
} else {  
    // the encoder is rotating in clockwise direction => increase the counter  
    counter++;  
    direction = DIRECTION_CW;  
}  
  
last_time = millis();  
}
```

III. Rotary encoder with LED (Control its brightness)

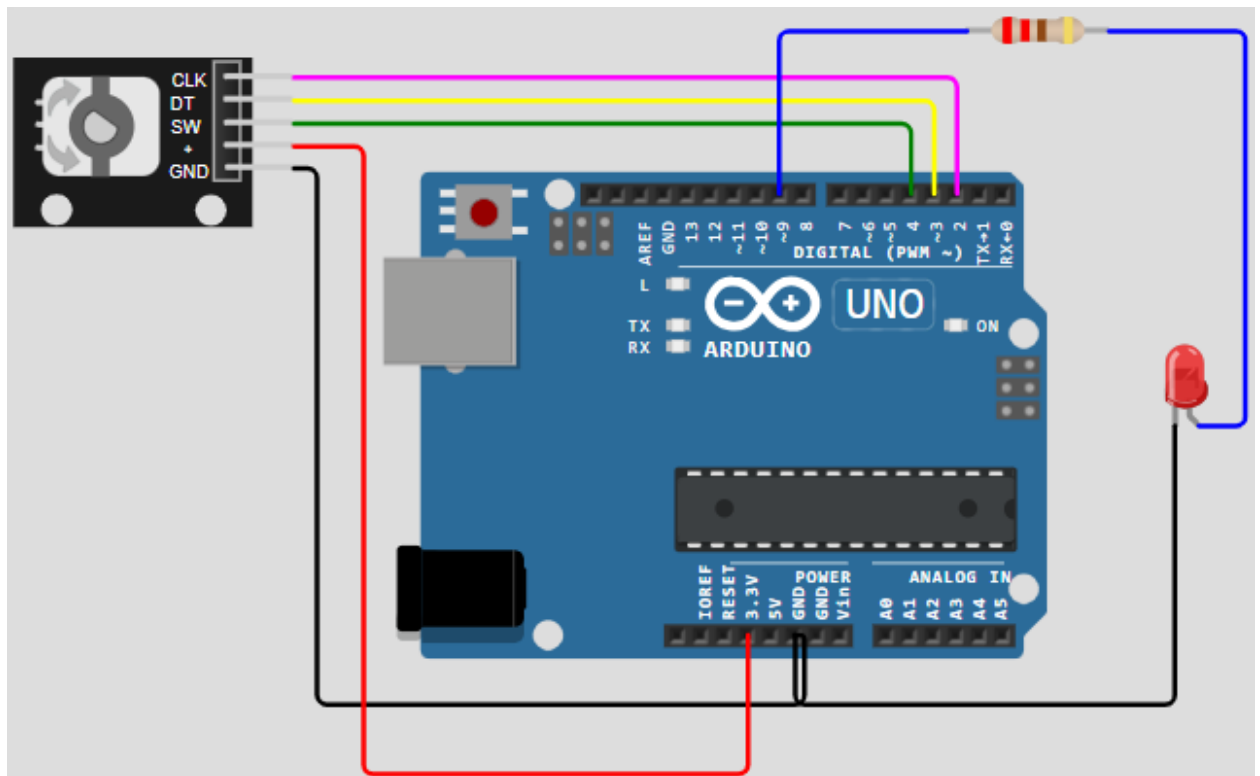


Figure 3: LED with rotary encoder circuit

```
#include <Servo.h>

#define CLK_PIN 2
#define DT_PIN 3
#define SW_PIN 4
#define LED_PIN 9

#define DIRECTION_CW 0 // clockwise direction
#define DIRECTION_CCW 1 // counter-clockwise direction

int counter = 0;
int direction = DIRECTION_CW;
int CLK_state;
int prev_CLK_state;
int brightness = 125; // middle value
```

```
void setup() {  
  Serial.begin(9600);  
  
  // configure encoder pins as inputs  
  pinMode(CLK_PIN, INPUT);  
  pinMode(DT_PIN, INPUT);  
  
  // read the initial state of the rotary encoder's CLK pin  
  prev_CLK_state = digitalRead(CLK_PIN);  
  pinMode(LED_PIN, OUTPUT);  
}  
  
void loop() {  
  
  // read the current state of the rotary encoder's CLK pin  
  CLK_state = digitalRead(CLK_PIN);  
  
  // If the state of CLK is changed, then pulse occurred  
  // React to only the rising edge (from LOW to HIGH) to avoid double count  
  if (CLK_state != prev_CLK_state && CLK_state == HIGH) {  
    // if the DT state is HIGH  
    // the encoder is rotating in counter-clockwise direction => decrease the counter  
    if (digitalRead(DT_PIN) == HIGH) {  
      direction = DIRECTION_CCW;  
      counter--;  
      brightness -= 10; // you can change this value  
    } else {  
      // the encoder is rotating in clockwise direction => increase the counter  
      direction = DIRECTION_CW;  
      counter++;  
    }  
  }  
}
```

```
    brightness += 10; // you can change this value
}

if (brightness < 0)
    brightness = 0;
else if (brightness > 255)
    brightness = 255;

// sets the brightness of LED according to the counter
analogWrite(LED_PIN, brightness);

Serial.print("COUNTER: ");
Serial.print(counter);
Serial.print(" | BRIGHTNESS: ");
Serial.println(brightness);
}

// save last CLK state
prev_CLK_state = CLK_state;
}
```

IV. Rotary encoder with servo motor (control servo motor according to rotary encoder's output)

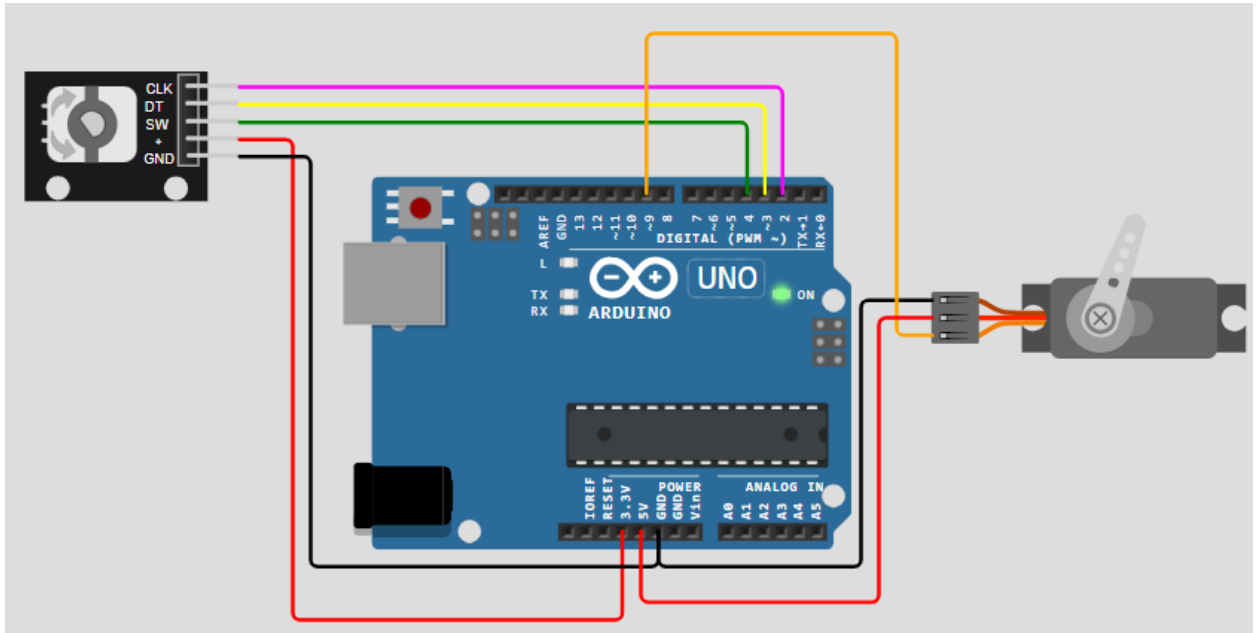


Figure 4: Rotary encoder controlling servo motor circuit

```
#include <Servo.h>

#define CLK_PIN 2
#define DT_PIN 3
#define SW_PIN 4
#define SERVO_PIN 9

#define DIRECTION_CW 0 // clockwise direction
#define DIRECTION_CCW 1 // counter-clockwise direction

int counter = 0;
int direction = DIRECTION_CW;
int CLK_state;
int prev_CLK_state;

Servo servo; // create servo object to control a servo
```

```
void setup() {  
  Serial.begin(9600);  
  
  // configure encoder pins as inputs  
  pinMode(CLK_PIN, INPUT);  
  pinMode(DT_PIN, INPUT);  
  
  // read the initial state of the rotary encoder's CLK pin  
  prev_CLK_state = digitalRead(CLK_PIN);  
  servo.attach(SERVO_PIN); // attaches the servo on pin 9 to the servo object  
  servo.write(0);  
}  
  
void loop() {  
  
  // read the current state of the rotary encoder's CLK pin  
  CLK_state = digitalRead(CLK_PIN);  
  
  // If the state of CLK is changed, then pulse occurred  
  // React to only the rising edge (from LOW to HIGH) to avoid double count  
  if (CLK_state != prev_CLK_state && CLK_state == HIGH) {  
    // if the DT state is HIGH  
    // the encoder is rotating in counter-clockwise direction => decrease the counter  
    if (digitalRead(DT_PIN) == HIGH) {  
      counter--;  
      direction = DIRECTION_CCW;  
    } else {  
      // the encoder is rotating in clockwise direction => increase the counter  
      counter++;  
      direction = DIRECTION_CW;  
    }  
  }  
}
```

```
}

Serial.print("DIRECTION: ");
if (direction == DIRECTION_CW)
    Serial.print("Clockwise");
else
    Serial.print("Counter-clockwise");

Serial.print(" | COUNTER: ");
Serial.println(counter);

if (counter < 0)
    counter = 0;
else if (counter > 180)
    counter = 180;

// sets the servo angle according to the counter
servo.write(counter);
}

// save last CLK state
prev_CLK_state = CLK_state;
}
```

2. MOTORS

Three very popular devices used to “make things (mechanical devices) move” include dc motors, RC servos, and stepper motors.

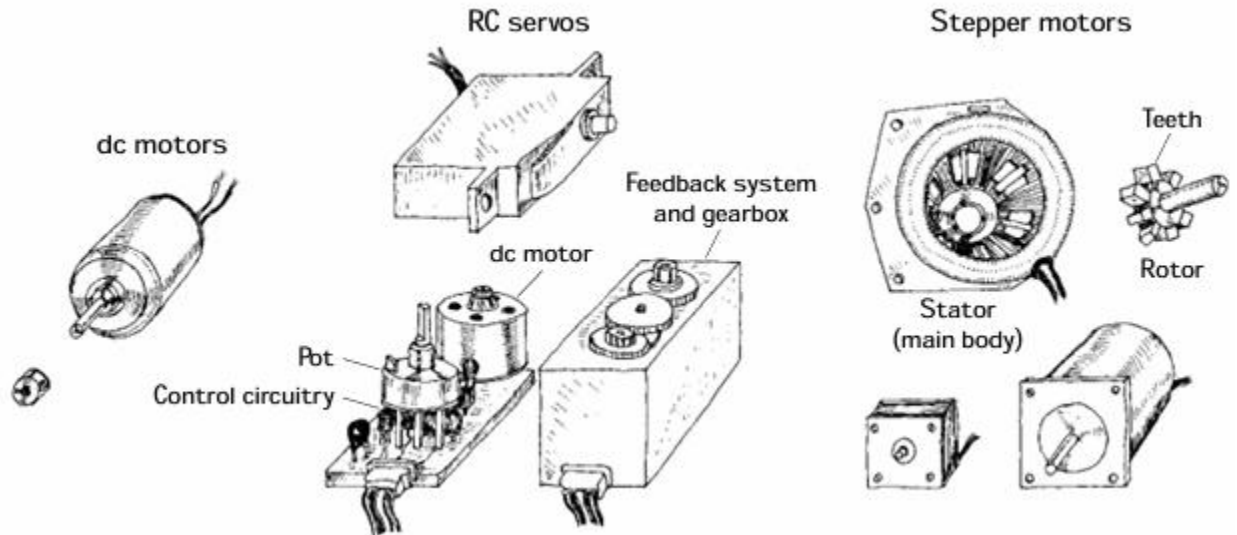


Figure 5: Types of motors

Difference between servo, stepper and DC motor:

Servo motor	Stepper motor	DC motor
Rotation is limited between 0^0 and 180^0	Rotation is unlimited	Rotation is unlimited
Rotating direction: clockwise, counter-clockwise	Rotating direction: clockwise, counter-clockwise	Rotating direction: clockwise, counter-clockwise

Rotating position: can be controlled to rotate to a specific angle between 0^0 and 180^0	Can be controlled to precisely rotate to any angle position	Very difficult to precisely rotate to a specific angle (Encoder is needed to control its position which uses closed-loop control method such as PID)
Rotating speed: can be controlled, but not smoothly	Can be controlled. Furthermore, acceleration and deacceleration can be controlled precisely	Easy to control how fast, but very difficult to precisely control to a specific speed value. (To control how many rounds per second, encoder is needed which uses PID method)
No external driver needed	Extra hardware driver required	Required
Extra power supply required	Extra power supply required	Required
Easy to control just with PWM signal	Complicated to control	Easily controlled using high-voltage PWM signal
Application: anything requiring only to rotate an angle between 0^0 and 180^0	Automation and robotics, printers, 3D printer, CNC, X-Y plotter, Cocktail machine	Used to control something that needs to rotate continuously but the value of speed does not need to be specified. For example fan, pump, RC car etc.

Motor selection guide:

1. Rotation to an angle between 0^0 and 180^0 use 180^0 servo motor
2. Precisely rotate to any position use stepper motor
3. Precisely rotate continuously without any position use DC motor or 360^0 servo motor

A. DC MOTOR

DC motors are simple two-lead, electrically controlled devices that come with a rotary shaft on which wheels, gears, propellers, etc., can be mounted. DC motors generate a considerable amount of revolutions per minute (rpm's) for their size and can be made to rotate clockwise or counterclockwise by **reversing the polarity** applied to the leads. At low speeds, dc motors provide little torque and minimal position control, making them obsolete for point like position-control applications. Most dc motors provide rotational speeds anywhere between **3000 and 8000 rpm** at a specific operating voltage typically set between **1.5 and 24 V**. The speed of a dc motor is most efficiently controlled by means of **pulse-width modulation**, whereby the motor is rapidly turned on and off. The width of the applied pulse, as well as the period between pulses, controls the speed of the motor. Also, it is worth noting that a freely running motor (no load) may draw little current (power). However, if a load is applied, the amount of current drawn by the motor's inner coils goes up immensely (up to 1000 percent or more).

➤ DC motor control (speed and direction) using L298N motor driver:

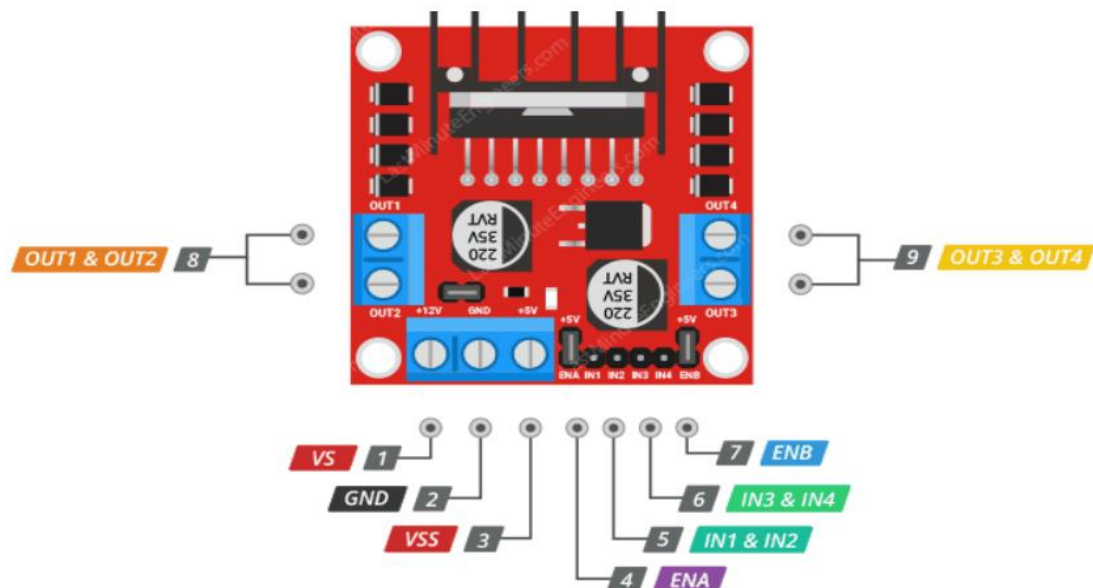


Figure 6: L298N pinout

i. 1 DC motor control

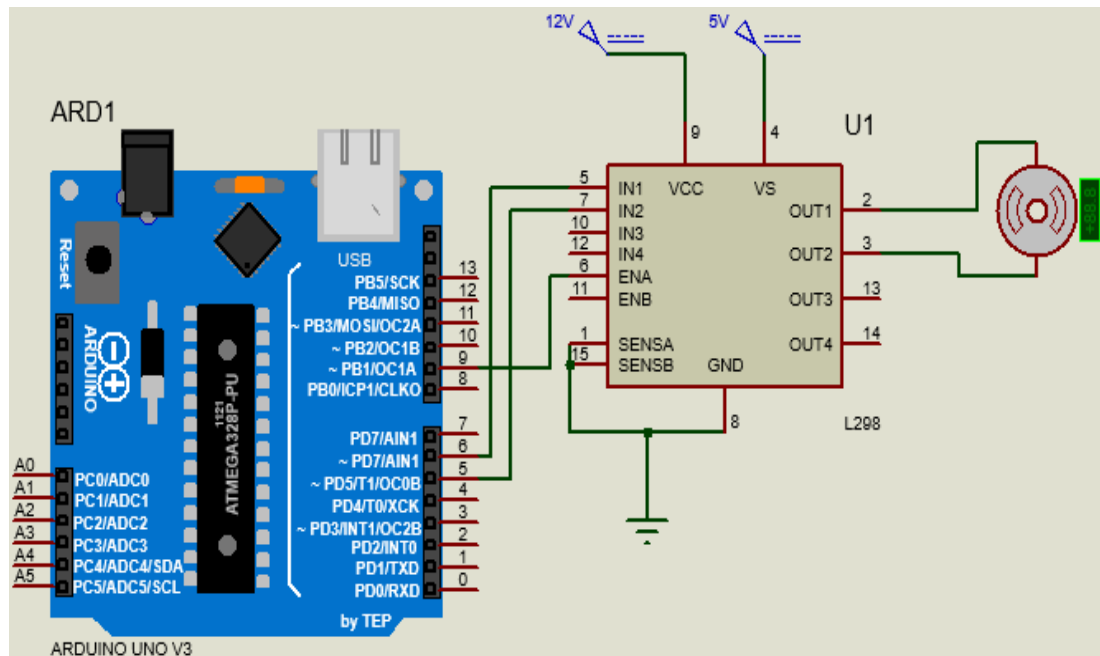


Figure 7: 1 dc motor control circuit

```
const int ENA_PIN = 9; // the Arduino pin connected to the EN1 pin L298N
const int IN1_PIN = 6; // the Arduino pin connected to the IN1 pin L298N
const int IN2_PIN = 5; // the Arduino pin connected to the IN2 pin L298N
```

```
// the setup function runs once when you press reset or power the board
```

```
void setup() {
  // initialize digital pins as outputs.a
  pinMode(ENA_PIN, OUTPUT);
  pinMode(IN1_PIN, OUTPUT);
  pinMode(IN2_PIN, OUTPUT);
}
```

```
// the loop function runs over and over again forever
```

```
void loop() {
  digitalWrite(IN1_PIN, HIGH); // control motor A spins clockwise
  digitalWrite(IN2_PIN, LOW); // control motor A spins clockwise

  for (int speed = 0; speed <= 255; speed++) {
    analogWrite(ENA_PIN, speed); // control the speed
    delay(10);
  }
}
```

```
delay(1000); // rotate at maximum speed 1 seconds in in clockwise direction

// change direction
digitalWrite(IN1_PIN, LOW); // control motor A spins anti-clockwise
digitalWrite(IN2_PIN, HIGH); // control motor A spins anti-clockwise

delay(1000); // rotate at maximum speed 1 seconds in in anti-clockwise direction

for (int speed = 255; speed >= 0; speed--) {
  analogWrite(ENA_PIN, speed); // control the speed
  delay(10);
}

delay(1000); // stop motor 1 second
}
```

ii. 2 DC motors control

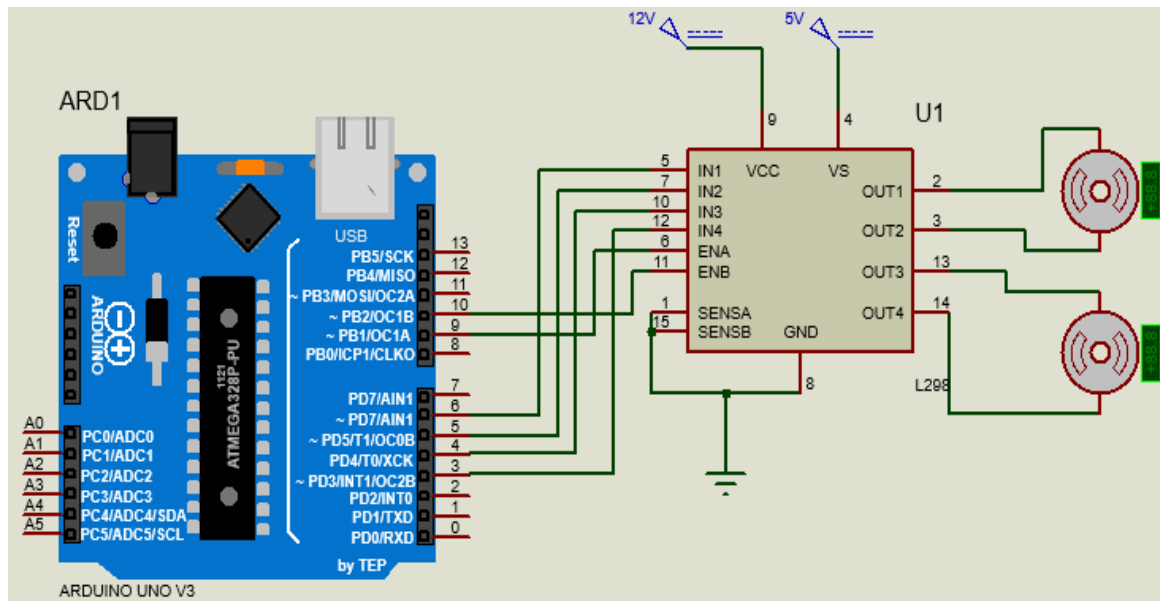


Figure 8: 2 dc motor control circuit

```
// Motor A pins
const int ENA_PIN = 9;
const int IN1_PIN = 6;
const int IN2_PIN = 5;

// Motor B pins
const int ENB_PIN = 10;
```

```
const int IN3_PIN = 4;
const int IN4_PIN = 3;

void setup() {
  // Motor A
  pinMode(ENA_PIN, OUTPUT);
  pinMode(IN1_PIN, OUTPUT);
  pinMode(IN2_PIN, OUTPUT);

  // Motor B
  pinMode(ENB_PIN, OUTPUT);
  pinMode(IN3_PIN, OUTPUT);
  pinMode(IN4_PIN, OUTPUT);
}

void loop() {
  // FORWARD (both motors same direction)
  digitalWrite(IN1_PIN, HIGH);
  digitalWrite(IN2_PIN, LOW);

  digitalWrite(IN3_PIN, HIGH);
  digitalWrite(IN4_PIN, LOW);

  // Ramp up speed
  for (int speed = 0; speed <= 255; speed++) {
    analogWrite(ENA_PIN, speed);
    analogWrite(ENB_PIN, speed);
    delay(10);
  }

  delay(1000);

  // REVERSE
  digitalWrite(IN1_PIN, LOW);
  digitalWrite(IN2_PIN, HIGH);

  digitalWrite(IN3_PIN, LOW);
  digitalWrite(IN4_PIN, HIGH);

  delay(1000);

  // Ramp down speed
  for (int speed = 255; speed >= 0; speed--) {
    analogWrite(ENA_PIN, speed);
```

```
    analogWrite(ENB_PIN, speed);  
    delay(10);  
}  
  
delay(1000);  
  
// STOP  
analogWrite(ENA_PIN, 0);  
analogWrite(ENB_PIN, 0);  
delay(1000);  
}
```

B. STEPPER MOTOR

Stepper motors, or steppers, are digitally controlled brushless motors that rotate a specific number of degrees (a step) every time a clock pulse is applied to a special translator circuit that is used to control the stepper. The number of degrees per step (resolution) for a given stepper motor can be as small as 0.72° per step or as large as 90° per step. Common general-purpose stepper resolutions are 15 and 30° per step. Unlike RC servos, steppers can rotate a full 360° and can be made to rotate in a continuous manner like a dc motor (but with a lower maximum speed) with the help of proper digital control circuitry. Unlike dc motors, steppers provide a large amount of torque at low speeds, making them suitable in applications where **low-speed and high-precision position control** is needed. They are used in printers to control paper feed and are used to help a telescope track stars. Steppers are also found in plotter- and sensor-positioning applications.

C. SERVO MOTOR

Servo motor is the device that can rotate its handle between 0° and 180° . It is used to control the angular position of an object. Servo motor includes 3 pins: VCC pin (+5V), GND pin (0V) and Signal pin (yellow or orange) which receives the PWM control signal from the microcontroller's pin. The angle is determined by the width of PWM signal. The angle is determined as follows:

If **PWM width = WIDTH_MIN**, servo rotates to 0° , if **PWM width = WIDTH_MAX**, servo rotates to 180° , and if PWM width is between WIDTH_MAX and WIDTH_MIN, servo motor rotates to angle between 0° and 180° in proportion.

The **MG996R** is a popular high-torque, metal-gear digital servo motor, often used in robotics and RC projects as an upgraded version of the MG995. It provides up to 13kg/cm of torque at 7.2V, operates within 4.8V–7.2V, and usually offers a 120-degree or 180-degree rotation range. It features improved centering and, due to its high stall current ($\sim 2.5\text{A}$), requires a strong external power supply, often controlled via a microcontroller.

I. CONTROLLED BY POTENTIOMETER

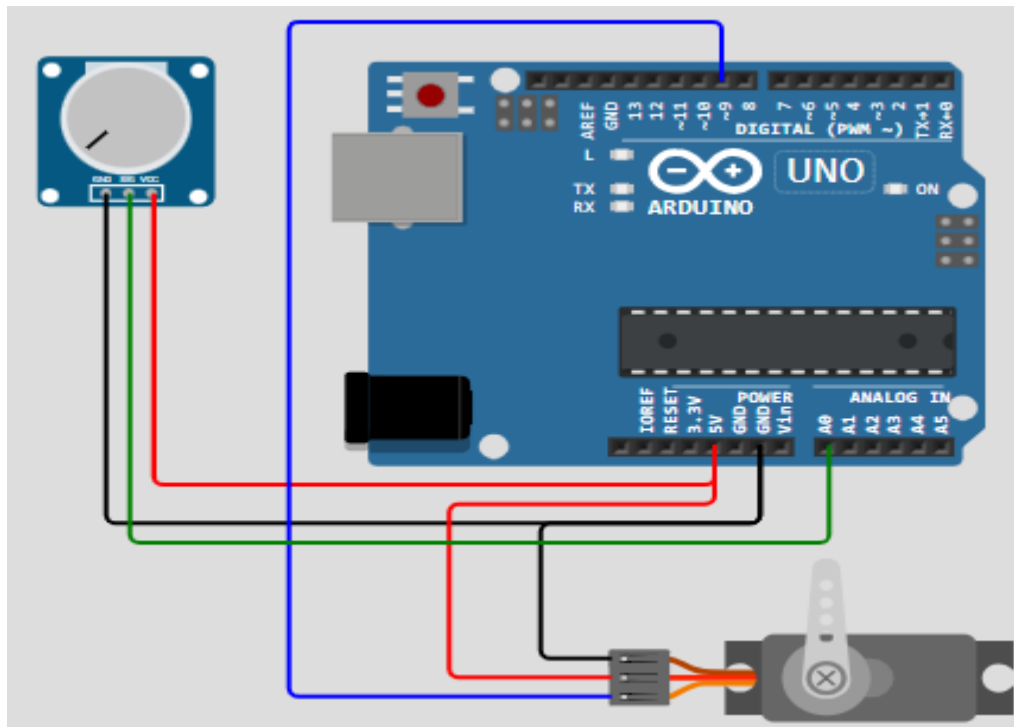


Figure 9: servo motor with potentiometer circuit

```
Servo myServo; // create servo object to control a servo
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
  myServo.attach(9); // attaches the servo on pin 9 to the servo object
}
void loop() {
  // reads the value of the potentiometer (value between 0 and 1023)
  int analogValue = analogRead(A0);
```

```
// scales it to use it with the servo (value between 0 and 180)
int angle = map(analogValue, 0, 1023, 0, 180);

// sets the servo position according to the scaled value
myServo.write(angle);

// print out the value
Serial.print("Analog: ");
Serial.print(analogValue);
Serial.print(", Angle: ");
Serial.println(angle);
delay(100);
}
```

II. POTENTIOMETER TRIGGERS SERVO MOTOR

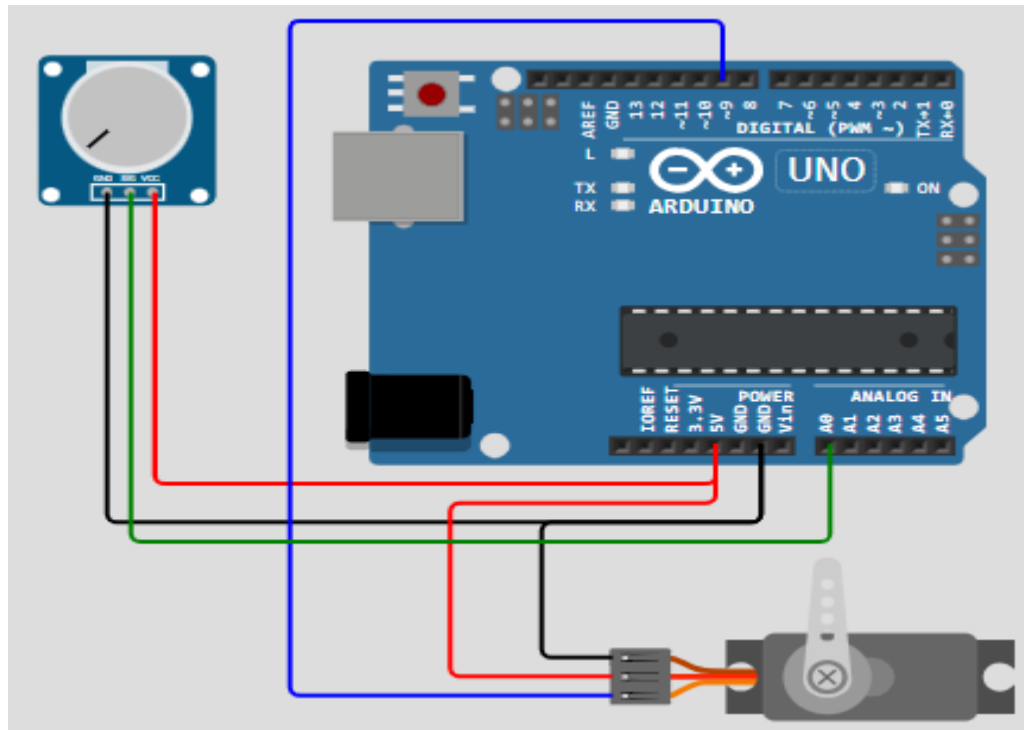


Figure 10: servo motor with potentiometer with and without threshold trigger

a) Analog threshold trigger

```
#include <Servo.h>
```

```
const int POTENTIOMETER_PIN = A0; // Arduino pin connected to Potentiometer pin
```

```
const int SERVO_PIN = 9; // Arduino pin connected to Servo Motor's pin
```

```
const int ANALOG_THRESHOLD = 500;
```

```
Servo servo; // create servo object to control a servo
```

```
void setup() {
```

```
    servo.attach(SERVO_PIN); // attaches the servo on pin 9 to the servo object
```

```
    servo.write(0);
```

```
}
```

```
void loop() {
```

```
int analogValue = analogRead(POTENTIOMETER_PIN); // read the input on analog
pin

if(analogValue > ANALOG_THRESHOLD)
    servo.write(90); // rotate servo motor to 90 degree
else
    servo.write(0); // rotate servo motor to 0 degree
}
```

Voltage threshold trigger

```
#include <Servo.h>

// constants won't change

const int POTENTIOMETER_PIN = A0; // Arduino pin connected to Potentiometer
pin

const int SERVO_PIN = 9; // Arduino pin connected to Servo Motor's pin

const float VOLTAGE_THRESHOLD = 2.5; // Voltages

Servo servo; // create servo object to control a servo

void setup() {
    servo.attach(SERVO_PIN); // attaches the servo on pin 9 to the servo object
    servo.write(0);
}

void loop() {
    int analogValue = analogRead(POTENTIOMETER_PIN); // read the input on
analog pin

    float voltage = floatMap(analogValue, 0, 1023, 0, 5); // Rescale to potentiometer's
voltage

    if(voltage > VOLTAGE_THRESHOLD)
        servo.write(90); // rotate servo motor to 90 degree
    else
        servo.write(0); // rotate servo motor to 0 degree
}
```

```
}  
  
float floatMap(float x, float in_min, float in_max, float out_min, float out_max) {  
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;  
}
```

PROJECT 1: BLUETOOTH CONTROLLED CAR USING L293D MOTOR DRIVER SHIELD

- L298N controls DC motors while L293D can control DC, Servo and Stepper motors
- L293D motor shield is based on the L293D IC and can drive 4 bi-directional DC motors, 2 stepper motors and 2 servo motors. It is mainly compatible with the Arduino UNO and MEGA boards. Each channel of this module has the maximum current of 1.2A and doesn't work if the voltage is more than 25v or less than 4.5v.
- The L293D is a dual-channel H-Bridge motor driver capable of driving a pair of DC motors or single stepper motor. This shield offers total four H-Bridges and each H-bridge can deliver up to 0.6A to the motor.

The shield also comes with a 74HC595 shift register that extends 4 digital pins of the Arduino to the 8 direction control pins of two L293D chips

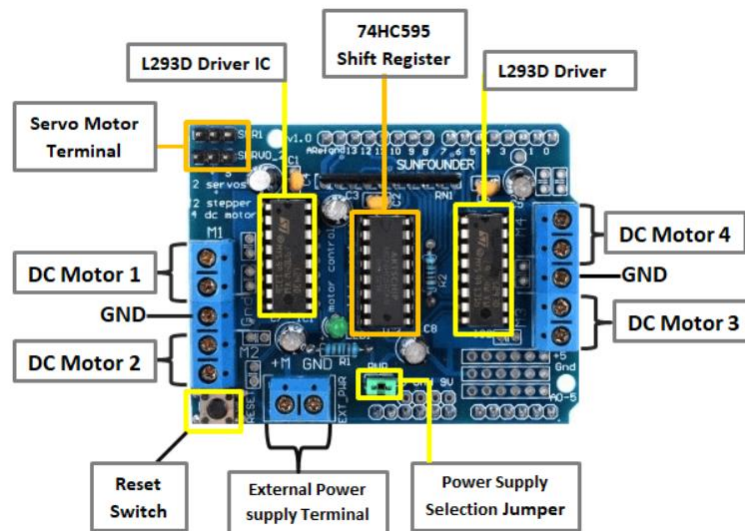


Figure 11: L293D

Power Supply considerations for the L293D motor driver:

You can power the Arduino through the USB and the motor powered off of a DC power supply or use two separate DC power supplies for the Arduino and motors. In this case make sure the jumper is removed from the motor shield.

Driving DC Motors: This motor shield can drive DC motors having voltages between 4.5 to 25V. The DC Motors I am using are rated at 9V. So, we will connect external 9V power supply to the EXT_PWR terminal. The motor is connected to any of M1, M2, M3 or M4 motor terminals. In this case, Arduino pin11 for M1, pin3 for M2, pin5 for M3, pin6 for M4 and pins 4, 7, 8 and 12 are all in use.

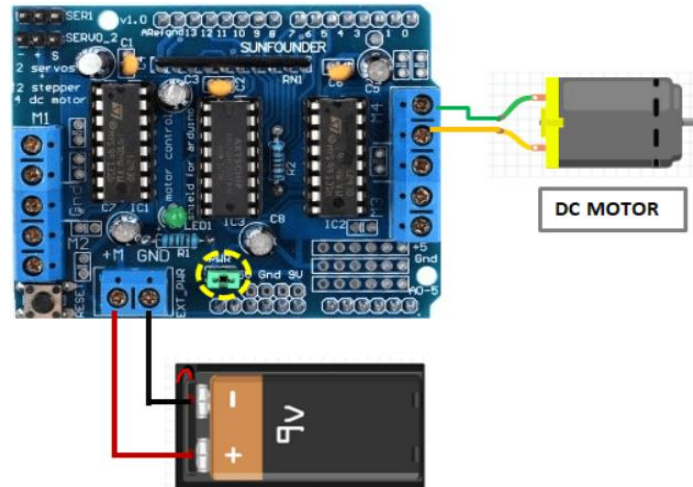


Figure 12: I293d with DC motor

```
#include <AFMotor.h>
AF_DCMotor motor(4);
void setup()
{
  //Set initial speed of the motor & stop
  motor.setSpeed(200);
  motor.run(RELEASE);
}
void loop()
{
  uint8_t i;
  // Turn on motor
  motor.run(FORWARD);
  // Accelerate from zero to maximum speed
  for (i=0; i<255; i++)
  {
```

```
    motor.setSpeed(i);
    delay(10);
}
// Decelerate from maximum speed to zero
for (i=255; i!=0; i--)
{
    motor.setSpeed(i);
    delay(10);
}
    // Now change motor direction
motor.run(BACKWARD);
// Accelerate from zero to maximum speed
for (i=0; i<255; i++)
{
    motor.setSpeed(i);
    delay(10);
}
// Decelerate from maximum speed to zero
for (i=255; i!=0; i--)
{
    motor.setSpeed(i);
    delay(10);
}
// Now turn off motor
motor.run(RELEASE);
delay(1000);
}
```

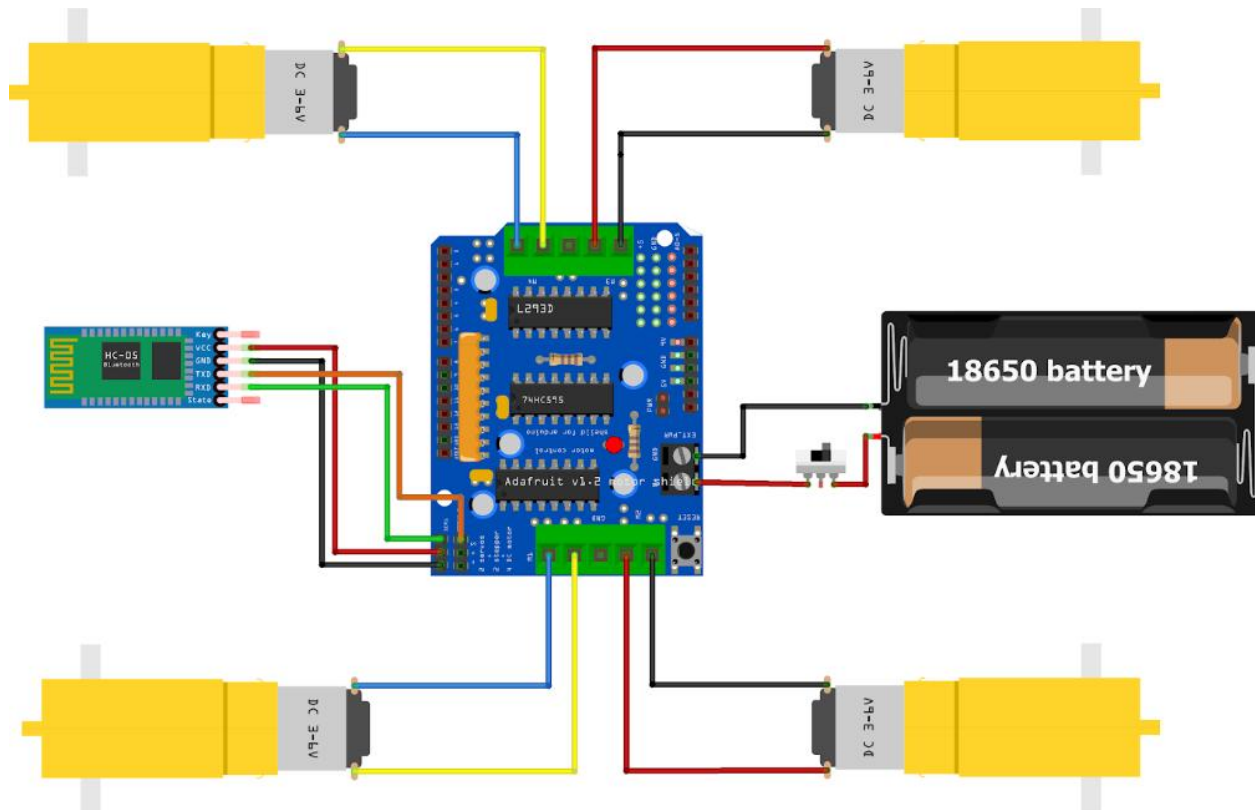


Figure 13: Bluetooth Controlled RC Car

```
#include <AFMotor.h>
#include <SoftwareSerial.h>

SoftwareSerial bluetoothSerial(9, 10); // RX, TX

//initial motors pin
AF_DCMotor motor1(1, MOTOR12_1KHZ);
AF_DCMotor motor2(2, MOTOR12_1KHZ);
AF_DCMotor motor3(3, MOTOR34_1KHZ);
AF_DCMotor motor4(4, MOTOR34_1KHZ);
char command;

void setup()
{
  bluetoothSerial.begin(9600); //Set the baud rate to your Bluetooth module.
}
```

```
void loop() {  
  if (bluetoothSerial.available() > 0) {  
    command = bluetoothSerial.read();  
  
    Stop(); //initialize with motors stoped  
  
    switch (command) {  
      case 'F':  
        forward();  
        break;  
      case 'B':  
        back();  
        break;  
      case 'L':  
        left();  
        break;  
      case 'R':  
        right();  
        break;  
    }  
  }  
}  
  
void forward()  
{  
  motor1.setSpeed(255); //Define maximum velocity  
  motor1.run(FORWARD); //rotate the motor clockwise  
  motor2.setSpeed(255); //Define maximum velocity  
  motor2.run(FORWARD); //rotate the motor clockwise  
  motor3.setSpeed(255); //Define maximum velocity  
  motor3.run(FORWARD); //rotate the motor clockwise  
  motor4.setSpeed(255); //Define maximum velocity
```

```
    motor4.run(FORWARD); //rotate the motor clockwise
}

void back()
{
    motor1.setSpeed(255); //Define maximum velocity
    motor1.run(BACKWARD); //rotate the motor anti-clockwise
    motor2.setSpeed(255); //Define maximum velocity
    motor2.run(BACKWARD); //rotate the motor anti-clockwise
    motor3.setSpeed(255); //Define maximum velocity
    motor3.run(BACKWARD); //rotate the motor anti-clockwise
    motor4.setSpeed(255); //Define maximum velocity
    motor4.run(BACKWARD); //rotate the motor anti-clockwise
}

void left()
{
    motor1.setSpeed(255); //Define maximum velocity
    motor1.run(BACKWARD); //rotate the motor anti-clockwise
    motor2.setSpeed(255); //Define maximum velocity
    motor2.run(BACKWARD); //rotate the motor anti-clockwise
    motor3.setSpeed(255); //Define maximum velocity
    motor3.run(FORWARD); //rotate the motor clockwise
    motor4.setSpeed(255); //Define maximum velocity
    motor4.run(FORWARD); //rotate the motor clockwise
}

void right()
{
    motor1.setSpeed(255); //Define maximum velocity
    motor1.run(FORWARD); //rotate the motor clockwise
```

```
motor2.setSpeed(255); //Define maximum velocity
motor2.run(FORWARD); //rotate the motor clockwise
motor3.setSpeed(255); //Define maximum velocity
motor3.run(BACKWARD); //rotate the motor anti-clockwise
motor4.setSpeed(255); //Define maximum velocity
motor4.run(BACKWARD); //rotate the motor anti-clockwise
}
void Stop()
{
motor1.setSpeed(0); //Define minimum velocity
motor1.run(RELEASE); //stop the motor when release the button
motor2.setSpeed(0); //Define minimum velocity
motor2.run(RELEASE); //rotate the motor clockwise
motor3.setSpeed(0); //Define minimum velocity
motor3.run(RELEASE); //stop the motor when release the button
motor4.setSpeed(0); //Define minimum velocity
motor4.run(RELEASE); //stop the motor when release the button
}
```

Driving Stepper Motors: If you are using 28BYJ-48 unipolar stepper, those motors are rated at 5V and offer 48 steps per revolution. So, connect external 5V power supply to the EXT_PWR terminal. Remember to remove the PWR jumper. The stepper motor is connected to any of the four terminals M1-M2(port1) and M3-M4(port2). In this experiment we are connecting it to Port 2.

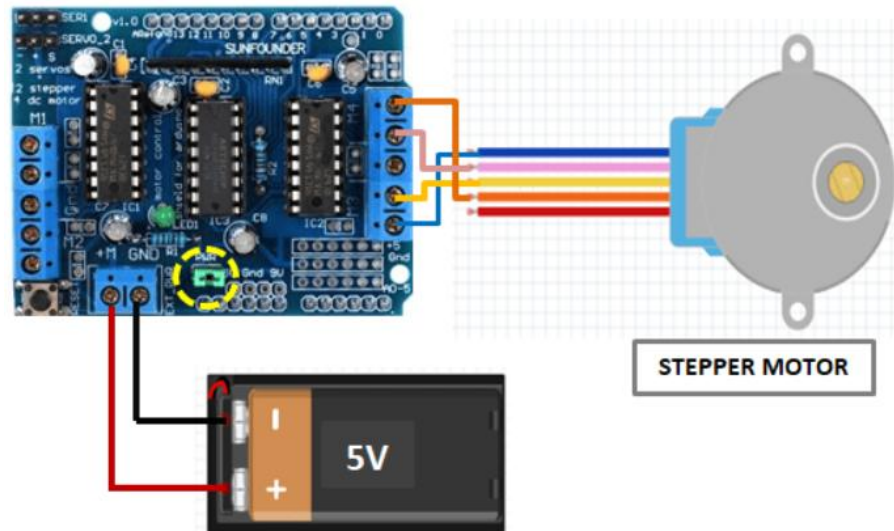


Figure 14: L293D with stepper motor

```
#include <AFMotor.h>

// Number of steps per output rotation

// Change this as per your motor's specification

const int stepsPerRevolution = 48;

// connect motor to port #2 (M3 and M4)

AF_Stepper motor(stepsPerRevolution, 2);

void setup() {

  Serial.begin(9600);

  Serial.println("Stepper test!");

  motor.setSpeed(10); // 10 rpm

}
```

```
void loop() {  
  
  Serial.println("Single coil steps");  
  
  motor.step(100, FORWARD, SINGLE);  
  
  motor.step(100, BACKWARD, SINGLE);  
  
  Serial.println("Double coil steps");  
  
  motor.step(100, FORWARD, DOUBLE);  
  
  motor.step(100, BACKWARD, DOUBLE);  
  
  Serial.println("Interleave coil steps");  
  
  motor.step(100, FORWARD, INTERLEAVE);  
  
  motor.step(100, BACKWARD, INTERLEAVE);  
  
  Serial.println("Microstep steps");  
  
  motor.step(100, FORWARD, MICROSTEP);  
  
  motor.step(100, BACKWARD, MICROSTEP);  
  
}
```

Driving Servo Motors: Driving servos with L293D motor shield is very easy. Just connect the three pins of the motor to the servo terminals of the shield. In this case Arduino pins 9, 10, 2 are in use and the power for the Servos comes from the Arduino's on-board 5V regulator, so you don't need an external power supply on the EXT_PWR terminal.

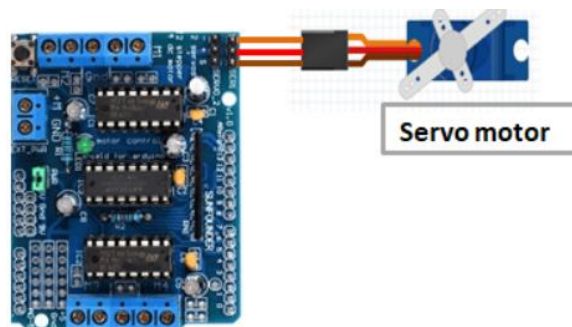


Figure 15: L293D with servo motor

```
#include <Servo.h>

Servo myservo; // create servo object to control a servo
int pos = 0; // variable to store the servo position

void setup()
{
  // attaches the servo on pin 10 to the servo object
  myservo.attach(10);
}

void loop()
{
  // sweeps from 0 degrees to 180 degrees
  for(pos = 0; pos <= 180; pos += 1)
  {
    myservo.write(pos);
    delay(15);
  }
  // sweeps from 180 degrees to 0 degrees
  for(pos = 180; pos >= 0; pos -= 1)
  {
    myservo.write(pos);
    delay(15);
  }
}
```